



NixChat

A secure desktop messenger in Java:
real-time chats, group roles, encrypted history

TEAM

Leonard Bronskiy · Khondamir Artikov
Roman Morgunov · Alisher Karavanov

REVIEWER

Dr. Shirin Noekhan
Fall 2026

WHY A NEW PROJECT

From PMC to NixChat

2ND SEMESTER

PMC Mission Control

- JavaFX app for a fictional military company
- Login, roles, shooting range
- Data lived in memory only

3RD SEMESTER

NixChat messenger

- A product people actually use
- Networking, threads, files, encryption
- Steps 1–5 carried over from PMC

WHAT WORKS TODAY

What is built so far



Real-time chat

Messages reach every client instantly over WebSocket



Encrypted history

Saved on disk with AES-256-GCM, survives restarts



Contacts

Everyone you talked to, sorted by last activity



Chats and roles

Owner, Admin, Member rules in the model, covered by tests

Next: accounts and login, private chats on the server, receipts, files, end-to-end encryption.

COMPANY STRUCTURE

Six departments, four positions each

Messaging

Private and group chats, the core product

`common.model`

Wallet

NixCoin virtual currency, transfers between friends

`common.wallet`

Music

Tracks in chats, shared playlists

`common.music`

Games

Games inside chats, starting with tic-tac-toe

`common.games`

Store

NixStore: install, search and rate mini-apps

`common.store`

Security

Encryption and account safety for all of them

`common.crypto`

In code: abstract **Department**, six subclasses, 24 positions; **NixChatCompany** lists them all.

ARCHITECTURE

Three modules, one shared model

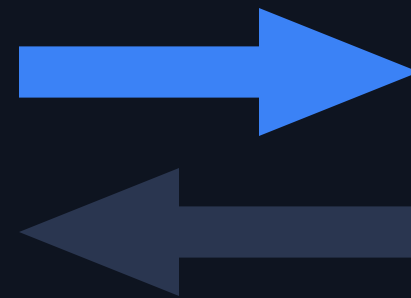
client

JavaFX 21 desktop app

ui · net · storage

3 local files, encrypted cache

WebSocket /ws



server

Spring Boot 4.1

ws · auth · api

Relays messages to clients

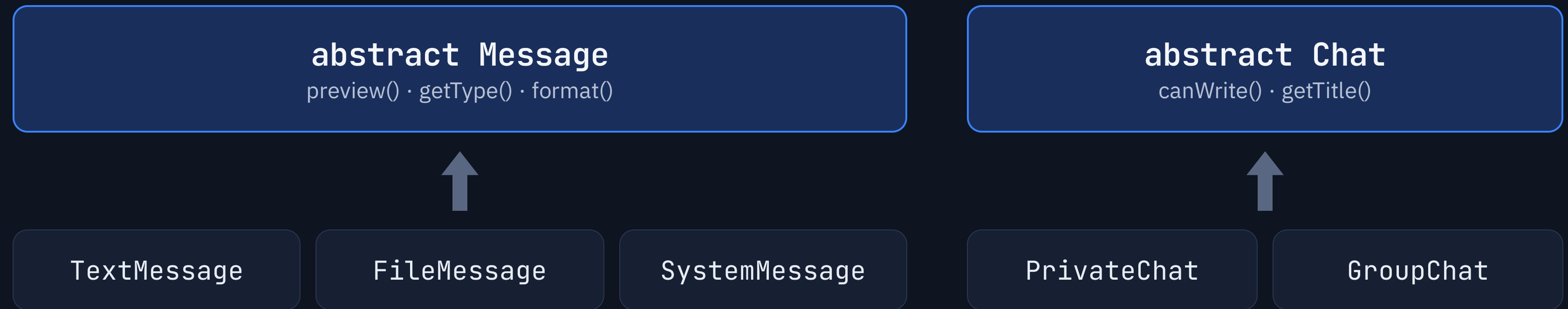
common Plain-Java domain model used by both sides: users, chats, messages, encryption, storage interfaces

TECHNOLOGY

Current, supported versions only

Layer	Technology	Why
Language	Java 21 (LTS)	Records, virtual threads
Server	Spring Boot 4.1.1	Supported to July 2027
Client	JavaFX 21 + AtlantaFX	Same GUI stack as PMC
Database	PostgreSQL, Redis	History, online status
Build and CI	Gradle, GitHub Actions	Tests on every push
License	Apache 2.0	Equal rights for all four

Two abstract classes, five subclasses



Interfaces: `Encryptor` (`AesGcmEncryptor`, `NoOpEncryptor`) and `MessageStorage` (`InMemoryMessageStorage`, `FileMessageStorage`)

One call, a different result for each type

```
List<Message> feed = List.of(  
    new TextMessage(chat, alice, "Hello!"),  
    new FileMessage(chat, alice, "photo.png", 1536),  
    new SystemMessage(chat, "bob joined"));  
  
for (Message m : feed)  
    System.out.println(m.preview());
```

OUTPUT

Hello!

photo.png (1.5 KB)

bob joined

Also polymorphic: the cache picks an **Encryptor** from settings, and **Chat.canWrite()** answers differently for private and group chats.

Three files, one per entity

`settings.properties`

User and settings

Username, server address, encryption on or off and its key

`AppSettings`

`contacts.csv`

Contact

One line per person: username, last seen, display name

`ContactStore`

`messages-cache.tsv`

Message

Every message type, contents encrypted with AES-256-GCM

`FileMessageStorage`

Each profile gets its own folder, so two clients can run side by side on one computer.

STEP 3 · EXCEPTIONS

A broken file never crashes the app

FOUR TYPES CAUGHT

Exception	When
IOException	Settings unreadable
NumberFormatException	Port is not a number
URISyntaxException	Bad server address
GeneralSecurityException	Wrong key, edited cache

```
MULTIPLE CATCHES + FINALLY
try (reader = Files.newBufferedReader(file)) {
    // read and decrypt each line
} catch (NoSuchFileException e) {
    // first launch: no cache yet
} catch (IOException e) {
    // disk error: start empty
} catch (RuntimeException e) {
    // unexpected: log the line
} finally {
    // print what was loaded
}
```

STEP 4 · GUI

Six screens, sketched for QA

01

Sign in

Username, password, error state

02

Create account

Validation rules shown inline

03

Chats

Chat list, feed, files, replies

04

New group

Name, members, read-only mode

05

Group info

Roles, promote, remove

06

Settings

Profile, server, encryption

Built with JavaFX like PMC, so every team member must be able to explain ChatView, MessageCell and ContactCell.

Each package has one job

```
uz.nixchat.common
```

```
├─ model
|   ├─ message
|   └─ chat
├─ crypto
└─ storage
```

```
uz.nixchat.client
```

```
├─ ui
├─ net
└─ storage
```

```
uz.nixchat.server
```

```
└─ ws
```

Cross-package calls from **NixChatApp**: `AppSettings.load()`, `WireFormat.decode()`, `Protocol.wsUrl()`, `ChatView.showMessage()`.

SECURITY

The server should only ever see ciphertext

DONE

Local history encrypted with AES-256-GCM; tampering is detected

NEXT

End-to-end: X25519 key exchange, passwords hashed with BCrypt

QUALITY

Every push is built and tested

32

unit tests

Messages · previews, sizes, validation

Chats · roles, kicking, read-only mode

Encryption · wrong key and tampering detected

Local files · survive restart, broken lines skipped

Departments · wallet, music, games, store

GitHub Actions runs the build and all tests on every push and pull request.

PLAN

The rest of the semester

October

- Accounts and login
- Private chats on the server
- History in PostgreSQL

November

- Groups with roles
- Receipts and typing
- File sharing

December

- End-to-end encryption
- Load test, deployment
- Final defense

LIVE DEMO

Thank you

Questions, then two clients chatting live

```
gradlew :server:bootRun  
gradlew :client:run --args="--profile=alice"  
gradlew :client:run --args="--profile=bob"
```